Quantum Computing @FII

Laborator 1

Instalare Python. Qiskit

În cadrul laboratoarelor de Quantum Computing, vom utiliza limbajul Python. Pentru a instala Python pe calculatoare puteţi utiliza următoarea pagină: `https://www.python.org/downloads/`.

După instalarea Python, trebuie sa verificati daca in linia de comanda puteţi rula comanda:

```
> python3
```

Dacă Python a fost instalat, cu ajutorul comenzii de mai sus veţi intra direct în consola Python, deci veti putea testa unele funcţionalităţi, să verificaţi o anumită sintaxă, sau să obţineţi documentaţia despre anumite module. Din consola Python se iese scriind *quit()* sau *exit()*. Pentru a rula un script.py din linia de comandă veti apela

```
> python3 script.py
```

Aşadar instalarea a reusit. În cazul în care nu merge sa rulaţi scriptul, trebuie să încercaţi să adaugati in variabila PATH calea catre executabilul python3. Pe Windows e posibil ca acest executabil sa se numească simplu python. Instrucţiuni despre cum se setează PATH gasiti aici:

1. `https://www.architectryan.com/2018/03/17/add-to-the-path-on-windows-10/`
2. `https://opensource.com/article/17/6/set-path-linux`

Dupa ce aţi setat variabila PATH veţi porni din nou linia de comandă si veti încerca sa rulaţi comanda.

La adresa `https://www.w3schools.com/python/` veţi găsi un tutorial ce vă poate fi de ajutor pentru a vă familiariza cu conceptele de bază ale programului Python.

Qiskit

Pentru a implementa circuite şi algoritmi din Quantum Computing, va trebui sa instalam un pachet Python pe care îl vom utiliza intensiv. Instalarea nu este chiar rapida, poate sa dureze mai mult in functie de calculatorul pe care il utilizati si de conexiunea la internet.

Astfel, în linia de comandă vom scrie

```
> pip3 install qiskit
```

Dupa ce ati instalat acest pachet, deschideţi în editorul preferat un fişier, să spunem test.py, cu următorul conţinut:

```
import qiskit
```

Apoi salvati fisierul si rulati comanda:

```
> python3 test.py
```

E posibil ca rularea sa dureze putin mai mult. Daca totul se incheie fara vreun cod de eroare in linia de comanda atunci instalarea s-a facut cu succes.

Pe partea de instalare qiskit vă poate fi de folos documentaţia care se găseţe pe site-ul `https://qiskit.org/` precum şi următorul tutorial `https://docs.quantum.ibm.com/guides/install-qiskit`.

Particularităţi de sintaxă - limbaj Python

- Indentarea în Python este mai mult decât parte a stilului de programare, este chiar parte din sintaxă
- Linia nouă termină o declarație, pentru a continua o declarație pe mai multe linii, se utilizează caracterul " \ ".
- După instrucțiuni nu este obligatoriu să puneți simbolul ';' precum în alte limbaje de programare, indentarea fiind suficientă.
- Un bloc de cod are toate liniile indentate cu același număr de spații (nu există begin și end sau). Instrucțiunile dintr-un bloc de cod vor fi grupate unele sub altele, pe același nivel de indentare.
- Exemplu: Afişarea numerelor de la 1 la 10.

```
n = 1
# Afisam numerele de la 1 la 10
while n <= 10:
    print(n)
    n += 1
print("Done")
```

- O singură linie se poate comenta în Python utilizând simbolul #. Pentru a comenta mai multe linii se utilizează trei ghilimele la inceputul, respectiv la sfârşitul textului.

```
# Comentariu
if a<10:
  ''' Comentariu '''
  print("Ok")
```

Tipuri de date şi variabile

- Python oferă tipuri de date numerice, booleene, șiruri (string, liste etc), dicționare, fișiere, clase, instanțe și excepții.

  Datele numerice sunt de 3 tipuri *int, float şi complex* (detalii)

```
a = 1        # a va fi de tip Integer
print(type(a))
#<class 'int'>
a = 1.5        # a va fi de tip Float
print(type(a))
#<class 'float'>
a=1-2j
print(type(a))
#<class 'complex'>
```

  Pentru lucrul cu caractere, Python oferă tipul *String*. Stringurile sunt incluse între ghilimele " " sau între ' '.

```
a = "Hello"   # a va fi de tip String
a[0]='h'
```

  Lista de elemente se declară utilizând paranteze drepte. O listă este o colecție ordonată de elemente, care poate conține elemente de diferite tipuri de date. Listările sunt definite între paranteze drepte `[]`.

```
lista1=[1,2,3]
lista2=['a','b','c']
lista3=['Hello', 30]
```

```
lista3[0]='Hello'
lista3[1]=30
```

Dicționare: Un dicționar este o structură de date neordonată, formată din perechi cheie-valoare, în care valoarea poate fi căutată folosind cheia. Cheile sunt obiecte *hashable*, ceea ce presupune că acestea trebuie să aibă o funcție *hash()* care returnează întotdeauna aceeași valoare pentru obiectul respectiv."

```
dict = {'Name': 'Ana', 'Age': 7, 'Class': 'First'}
print ("dict['Name']: ", dict['Name'])
#dict['Name']:  Ana
#dict['Age']=8.   #modifica in dictionar
```

Tipuri de date booleene: bool - Reprezintă o valoare logică, care poate fi fie True (adevărat), fie False (fals).

```
x = True
y = False
```

Structuri repetitive

- Instrucţiunea **for** iterează după elementele unei secvenţe:

```
for elem in secvecnta:
    instructiuni
```

- Exemplu

```
for i in range(0, 10, 2):  # Itereaza de la 0 la 8, cu pasul 2
    print(i)
```

sau

```
cuvant = "Python"
for litera in cuvant:
    print(litera)
```

- Instructiunea **while** continuă iterația până când condiția este adevărată

```
while conditie:
    instructiuni
```

- Exemplu:

```
numar = -1  # Initializam cu un numar negativ pentru a intra in bucla
while numar <= 0:
    numar = int(input("Introdu un numar strict pozitiv: "))
print(f"Numarul introdus este: {numar}")
```

Structuri condiționale

- În Python, if este o structură de control condițională care permite executarea unui bloc de cod doar dacă o anumită condiție este adevărată (True). Dacă condiția este falsă (False), Python va sări peste acel bloc de cod și va continua execuția programului.

```
if conditie1:
    instructiuni
elif conditie2:
    instructiuni
else:
instructiuni
```

- Pot exista mai multe secțiuni elif sau nici una, iar secțiunea else apare o dată sau niciodată.
- Exemplu:

```
 if numar > 0:
    print("Numarul este pozitiv!")
elif numar == 0:
    print("Numarul este zero!")
else:
    print("Numarul este negativ.")
```

Funcţii

- Funcţiile sunt definite utilizând cuvântul cheie **def** urmat de numele funcţiei, și de parantezele rotunde, în care se pot trece parametrii (argumente). Dacă nu se include return sau se execută return fară argumente, atunci funcția va întoarce None.
- Exemplu:

```
def suma(a,b):
  return a+b
result=suma(1,2)
print(result)
#3
```

- În Python se pot include funcții, variabile și clase definite în alte fișiere. Un astfel de fișier, ce poate fi inclus, poartă denumirea de modul (detalii). În Quantum computing vom utiliza modulele **math**, **random** şi **cmath**.

  Modulele **math** şi **cmath** ne furnizează multe funcţii şi metode de a lucra cu numerele reale, respectiv, cu numerele complexe

```
import cmath
cmath.sqrt(-1)
#1j
z=complex(1,-1)
print(z)
#(1-1j)
z.real
#1.0
cmath.e
#2.718281828459045
```

```
import random
random.randint(1,10)
#genereaza un num\u ar \^ intreg din 1,10
#4
```

Atunci când includem modulul putem să îl folosim cu un alt nume, eventual, pentru a evita coliziuni de denumiri:

```
import random as rand
rand.random()
#0.4302922009221001
```

Exerciţii propuse:

1. Scrieţi un script în Python care afişează toţi multiplii de 3 din intervalul $2 : 50$.
2. Scrieţi un script în Python care verifică dacă un număr este par sau nu.
3. Scrieţi un program care calculează media valorilor unui vector de lungime dată.
4. Realizaţi un script care afişează răsturnatul unui număr întreg, dat de la tastatură.
5. Scrieţi un program în Python care calculează $n!$ unde n este dat de la tastatura.
6. Realizaţi un script ce calculează adunarea, respectiv înmulţirea a două matrici.
7. Având date de la tastatură valorile întregi, a, b şi c, să se calculeze rădăcinile ecuaţiei $ax^2+bx+c = 0$.